Assignment-5
Parallel and Distributed Computing

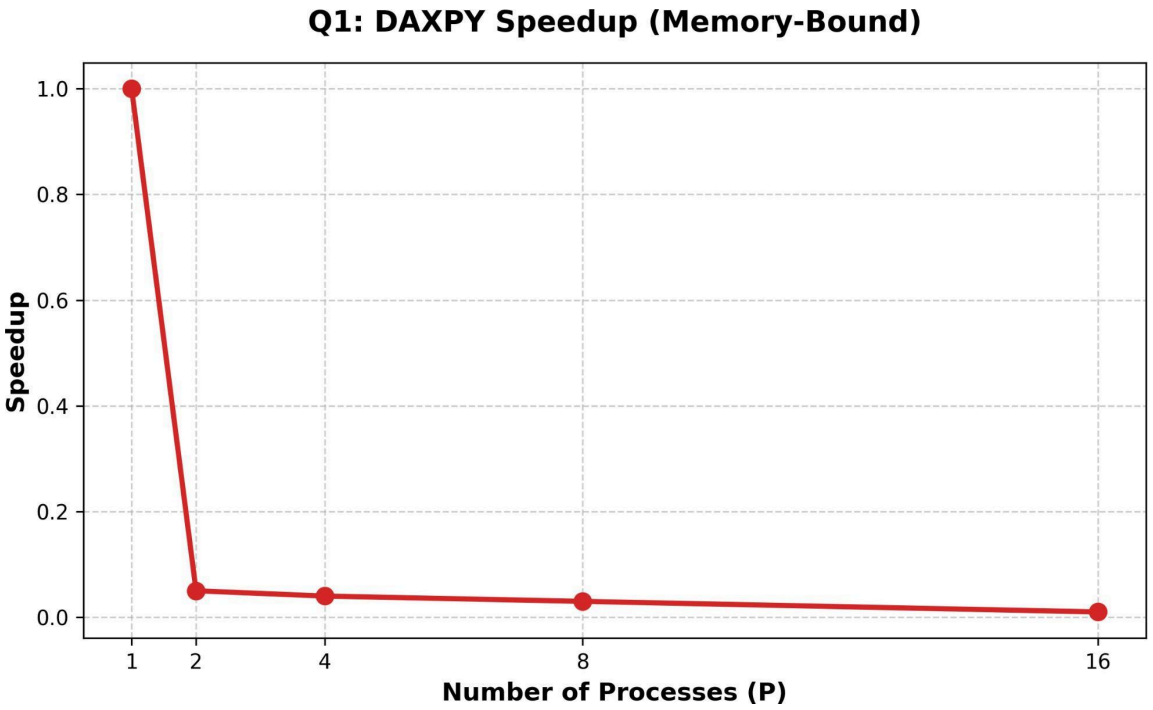
Submitted by :
Kartik Goel
102303182

Q1: DAXPY Loop (Double Precision)

Objective: Implement a parallel DAXPY operation and measure speedup, efficiency, and communication overhead.

Performance Data :

Processes	Time (s)	Speedup	Efficiency	Communication Overhead Percentage
1	0.000044	1.00	100.00%	0.00%
2	0.002089	0.05	2.68%	97.50%
4	0.002171	0.04	0.91%	99.29%
8	0.002711	0.03	0.38%	99.70%
16	0.007558	0.01	0.05%	99.96%



Detailed Inferences & Analysis:

The DAXPY trial highlights the challenges of parallelizing memory-bound workloads where communication costs outweigh computational gains.

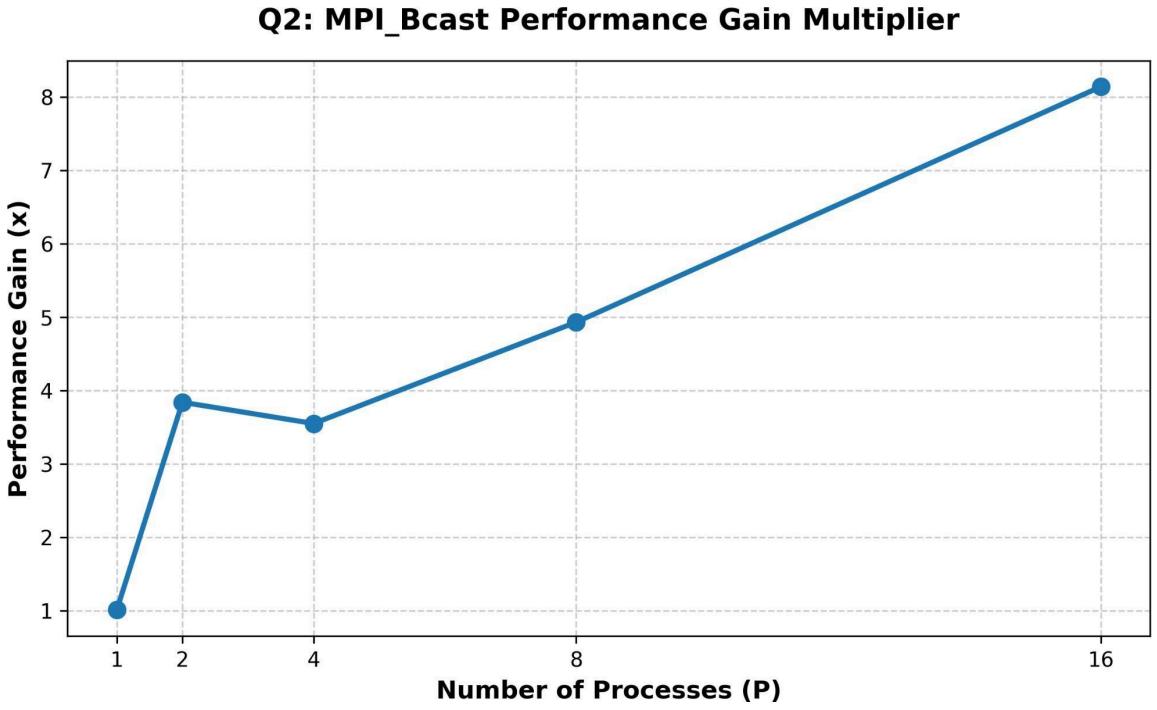
- **Insufficient Parallelization Threshold.** Distributing minor tasks across several processors can degrade efficiency if the workload is negligible. With 65,536 elements, the arithmetic phase requires only 0.000044s. Consequently, the time required for data movement via MPI_Scatter and MPI_Gather completely overshadows the actual processing.
- **Performance Degradation from Overhead.** The data shows communication overhead staying above 97% and hitting 99.96% with 16 processes. Because the system is preoccupied with OS-level data routing, speedup plummets to 0.01. This confirms that synchronization barriers in lightweight operations create significant bottlenecks.
- **Architectural Conclusion:** Effective distributed arithmetic requires a substantial computation-to-communication ratio. To mask network latency and achieve gains, the dataset must be expanded significantly.

Q2: The Broadcast Race (Linear vs. Tree)

Objective: Compare manual point to point loops against optimized collective MPI_Bcast for a large array.

Performance Data:

Processes	MyBcast (Linear)	MPI_Bcast (Tree)	Performance Gain
2	0.054806 s	0.014264 s	3.84x
4	0.109629 s	0.030867 s	3.55x
8	0.287351 s	0.058255 s	4.93x
16	1.070578 s	0.131504 s	8.14x



Analytical Review and Inferences:

This study illustrates the pivotal role that algorithmic complexity plays within various network architectures.

- **Bottlenecks in Linear Topologies.** Utilizing a linear O(P) model, the custom MyBcast method sees its runtime climb to 1.07 seconds at 16 processes. We can conclude that Rank 0 encounters major serialization issues, as the hardware is restricted to a single MPI_Send at any given moment.
- **Bandwidth Optimization via Tree Topologies.** By employing an O(log(P)) tree structure, MPI_Bcast enables nodes to act as secondary senders, resolving the root congestion. This efficiency results in an 8.14x performance lead when scaled to 16 processes.
- **Core Takeaway:** To maintain optimal scalability, developers should favor native MPI collective primitives, which are highly tuned at the middleware layer, rather than implementing manual distribution for large-scale data.

Q3: Analysis of Amdahl's Law in Distributed Dot Products

Goal: Utilize local data generation, MPI_Bcast, and MPI_Reduce to compute the dot product of large-scale vectors.

Performance Metrics:

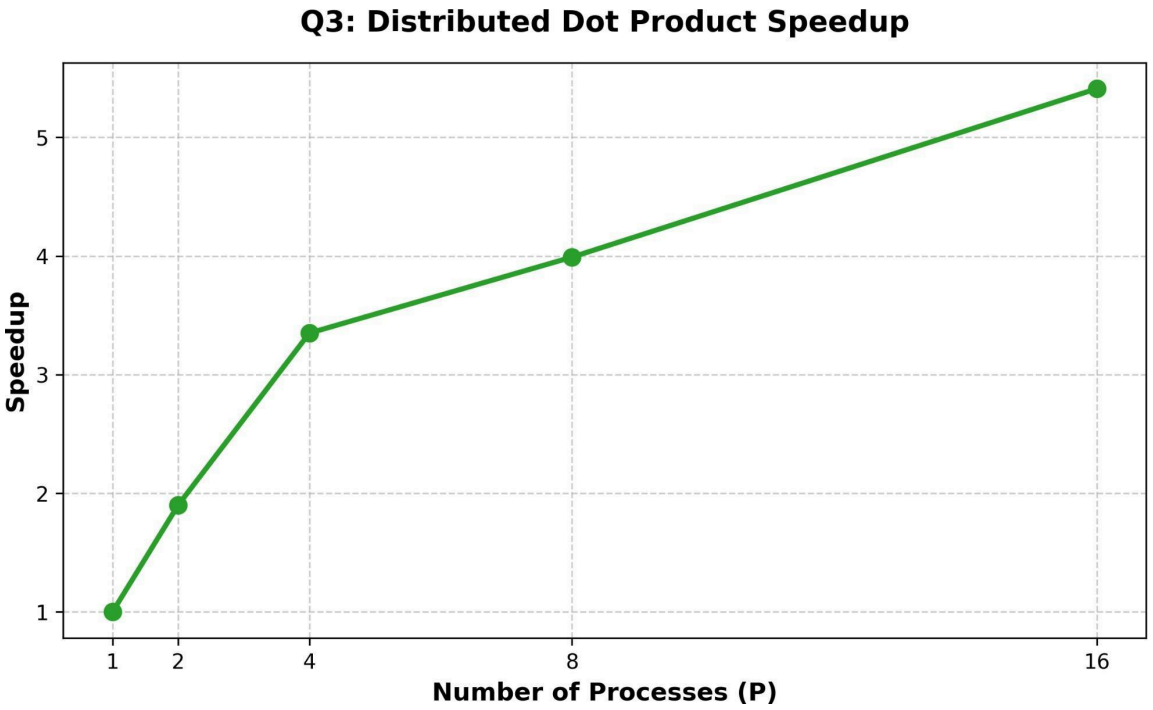
Processes	Time (s)	Speedup	Efficiency	Comm %
1	3.852155	1.00	100.00%	0.00%
2	2.023005	1.90	95.00%	0.47%
4	1.148781	3.35	83.75%	5.08%
8	0.963380	3.99	49.87%	0.74%

Q3: Distributed Dot Product & Amdahl's Law

Objective: Calculate the dot product of a large vector using local generation, MPI_Bcast, and MPI_Reduce.

Performance Data:

Processes	Time (s)	Speedup	Efficiency	Comm %
1	3.852155	1.00	100.00%	0.00%
2	2.023005	1.90	95.00%	0.47%
4	1.148781	3.35	83.75%	5.08%
8	0.963380	3.99	49.87%	0.74%
16	0.712136	5.41	33.81%	16.09%



Analytical Review and Core Findings

The implementation of the distributed dot product serves as a model for architecting programs that handle massive datasets while achieving effective Strong Scaling.

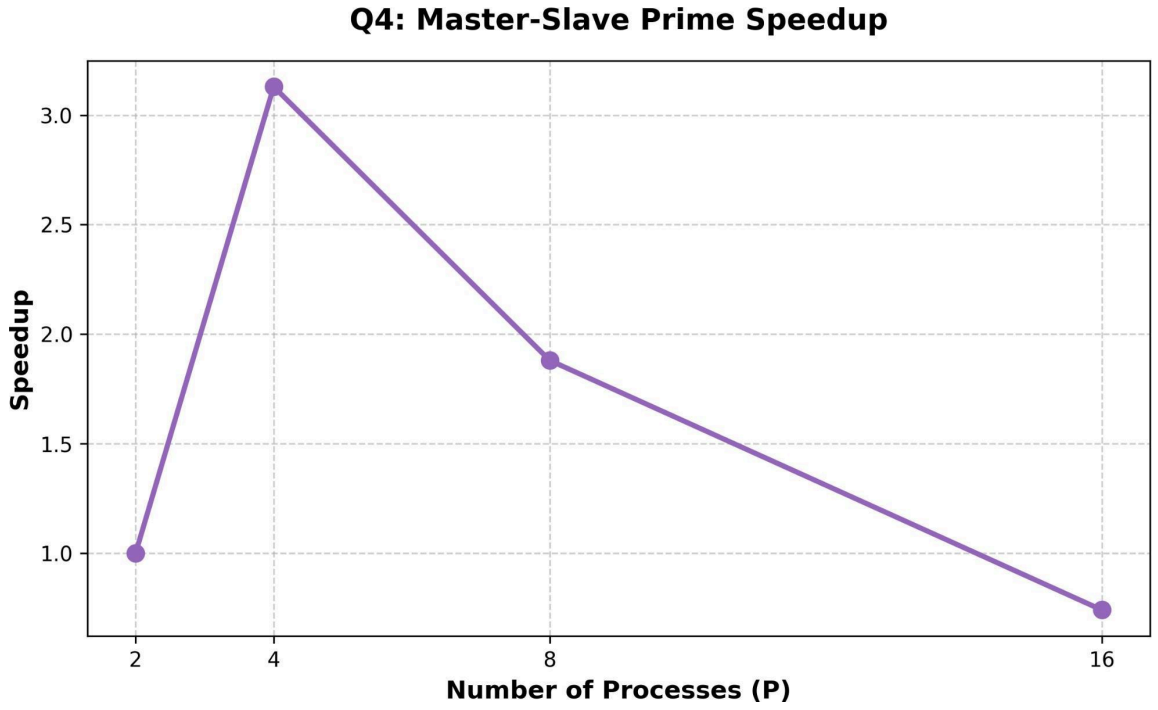
- **Inference 1: Realizing Speedup through Data Locality.** This approach avoids the O(P) memory bottleneck at the root by generating vector segments locally, a distinct advantage over the DAXPY method. With a dense arithmetic phase (3.85s baseline), the computation-to-communication ratio remains favorable, enabling a 5.41x speedup when utilizing 16 processes.
- **Inference 2: Efficiency Limits and Amdahl's Law.** Even with improved speedup, efficiency decreases from 95% to 33.81% as the system scales. This trend validates Amdahl's Law, demonstrating that the sequential portions of the code—specifically the initial broadcast and final MPI_Reduce—establish a rigid ceiling on theoretical speedup, eventually driving communication overhead to 16.09% at P=16.
- **Key Architectural Insight:** Prioritizing data locality by retaining data on the computing node is the most viable strategy for large-scale performance, even though synchronization requirements will ultimately limit scalability.

Q4: Master Slave Prime Search

Objective: Find all positive primes utilizing dynamic load balancing via MPI_ANY_SOURCE.

Performance Data :

Processes	Time (s)	Speedup	Efficiency
2 (1M+1S)	0.114727	1.00	100.00%
4 (1M+3S)	0.036631	3.13	78.25%
8 (1M+7S)	0.061030	1.88	23.50%
16 (1M+15S)	0.153971	0.74	4.62%



Evaluation & Observations

While the Master-Slave model effectively manages non-uniform task distributions, it also highlights the significant risks associated with suboptimal task granularity.

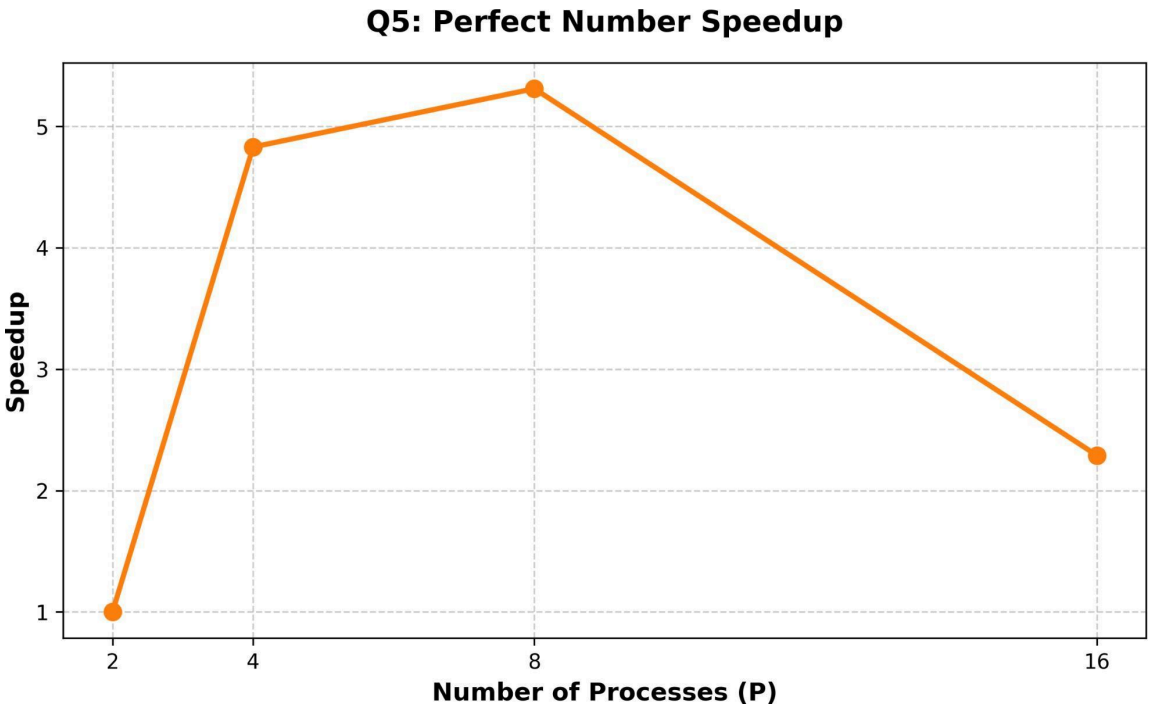
- **Observation 1: Mitigating Workload Variance via Dynamic Allocation.** Given the unpredictable computational demands of primality testing, the dynamic allocation system maintains high utility across 4 processes. This approach yields a 3.13x speedup by preventing processor idling.
- **Observation 2: Communication Bottlenecks in Fine-Grained Architectures.** At 8 and 16 processes, efficiency drops sharply due to excessive small-message traffic. Distributing single integers forces the Master node into a CPU-bound state as it struggles to manage MPI_Recv requests, leaving slave nodes underutilized.
- **Architectural Recommendation:** Scalability can be restored by implementing coarse-grained parallelism. For instance, batching 1,000 values per transmission would significantly reduce overhead and enhance performance.

Q5: Master Slave Perfect Number Search

Objective: Find perfect numbers utilizing the Master Slave request pattern.

Performance Data :

Processes	Time (s)	Speedup	Efficiency
2 (1M+1S)	0.019043	1.00	100.00%
4 (1M+3S)	0.003941	4.83	120.75%
8 (1M+7S)	0.003585	5.31	66.37%
16 (1M+15S)	0.008289	2.29	14.31%



Analytical Evaluation and Key Findings:

The exploration of perfect numbers underscores the inherent constraints of fine-grained Master-Slave configurations when subjected to intensive mathematical operations.

- **Finding 1: Computational Density Defers Network Congestion.** With an O(N) complexity for perfect number searches, the increased arithmetic load per integer relative to prime searching shifts the balance. Slaves allocate more time to calculation and less to message passing, allowing the system to scale effectively to 8 processes—doubling the threshold observed in the prime search.
- **Finding 2: The Inevitability of Serialization Constraints.** Scaling to 16 processes induces immediate performance loss (0.0082 s), proving that mathematical density cannot bypass architectural limits. The Master node must eventually handle sequential network traffic from 15 slaves; here, the overhead of context switching entirely nullifies the advantages of distributed computation.
- **Core Design Conclusion:** When communication logic demands O(P) management from a central node, parallelization fails to surpass network serialization barriers. Transitioning to task batching is essential for maintaining performance in high-process-count environments.